

Absolutely. One small terminology correction first: `resource`, `data`, and `provider` **are Terraform configuration blocks**, not all “attributes.” Attributes are things inside blocks, like `name = "my-vpc"`.

Here’s a beginner-friendly **Terraform cheat sheet of the important blocks and concepts**.

1. The 3 most important ones

`resource` — Terraform manages something

```
hcl

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}
```

Think:

"Terraform owns/manages this."

Usually Terraform creates it:

Terraform → AWS → Create VPC → Manage VPC

But you can also import an existing resource:

```
Existing AWS VPC
  ↓
terraform import
  ↓
Terraform manages it
```

`data` — Terraform reads something that already exists

```
hcl

data "aws_vpc" "existing" {
  id = "vpc-123456"
}
```

Think:

"Don't create this. Just give me information about it."

For example, use an existing VPC:

```
hcl

resource "aws_subnet" "app" {
  vpc_id      = data.aws_vpc.existing.id
  cidr_block = "10.0.1.0/24"
}
```

So:

```
Existing VPC
  ↓
data
  ↓
Get VPC information
  ↓
Create subnet inside it
```

provider — tells Terraform how to talk to a platform

```
hcl

provider "aws" {
  region = "ap-south-1"
}
```

Think:

"How should Terraform connect/talk to AWS?"

Providers exist for many platforms:

```
AWS
Azure
Google Cloud
Kubernetes
GitHub
Cloudflare
Databases
etc.
```

Example:

```
hcl
```

```
provider "aws" {  
  region = "ap-south-1"  
}
```

2. variable — input to your Terraform code

This is like a parameter/function argument.

```
hcl  
  
variable "environment" {  
  type    = string  
  default = "dev"  
}
```

Then:

```
hcl  
  
resource "aws_s3_bucket" "app" {  
  bucket = "my-app-${var.environment}"  
}
```

You can provide:

```
Bash  
  
terraform apply -var="environment=prod"
```

Think:

variable = something I want the person running Terraform to provide/configure.

Common types:

```
hcl  
  
string  
number  
bool  
list  
map  
object  
set
```

3. `output` — show a value after Terraform runs

For example:

```
hcl

output "vpc_id" {
  value = aws_vpc.main.id
}
```

After:

```
Bash

terraform apply
```

you might see:

```
vpc_id = "vpc-123456"
```

Think:

`output` = **give this information back to me.**

Very common for:

- VPC ID
- Load balancer URL
- EC2 IP
- EKS cluster name
- resource IDs

4. `locals` — reusable calculated values

Example:

```
hcl

locals {
  environment = "prod"

  common_tags = {
    Environment = "prod"
    Project     = "my-app"
  }
}
```

Then:

```
hcl

resource "aws_s3_bucket" "app" {
  bucket = "my-app-${local.environment}"

  tags = local.common_tags
}
```

Think:

`locals` = **variables calculated/defined inside Terraform.**

Difference:

variable → input from outside

local → internal reusable value

5. `module` — reuse Terraform code

Suppose you have:

```
modules/
├─ vpc/
├─ eks/
└─ secrets/
```

You can use a module:

```
hcl

module "vpc" {
  source = "../modules/vpc"

  environment = "prod"
  cidr_block  = "10.0.0.0/16"
}
```

Think:

`module` = **reusable Terraform package/code.**

This is extremely common in real projects.

For example:

```
root Terraform
├──
├── VPC module
├── EKS module
├── RDS module
└── Secrets module
```

6. `terraform` block — Terraform's own configuration

Example:

```
hcl

terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

Think:

`terraform` = **configuration about Terraform itself.**

It can specify:

- Terraform version
- providers
- provider versions
- backend/state configuration
- required features

7. `backend` — where Terraform state is stored

Usually inside the `terraform` block:

```
hcl

terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "ap-south-1"
  }
}
```

Think:

Backend = where Terraform's state lives.

Without a remote backend, your state may be local:

```
terraform.tfstate
```

With a remote backend:

```
Terraform
  ↓
Remote state
  ↓
S3
```

In teams, remote state is extremely common.

8. `moved` — tell Terraform a resource was renamed/moved

Suppose you had:

```
hcl

resource "aws_s3_bucket" "old" {
  ...
}
```

and rename it:

```
hcl

resource "aws_s3_bucket" "new" {
  ...
}
```

Terraform might think:

```
Delete old  
Create new
```

But you actually just renamed it.

You can tell Terraform:

```
hcl  
  
moved {  
  from = aws_s3_bucket.old  
  to   = aws_s3_bucket.new  
}
```

Think:

`moved` = **this is the same resource; I just changed its Terraform address.**

9. `import` — bring an existing resource under Terraform management

This is slightly different from the `data` block.

Example:

```
AWS  
└─ Existing VPC  
    │  
    │ terraform import  
    ↓  
Terraform state  
    │  
    ↓  
resource "aws_vpc" "main"
```

Modern Terraform also supports an `import` block:

```
hcl  
  
import {  
  to = aws_vpc.main
```



```
id = "vpc-123456"
```

Think:

`import` = **this already exists, and I want Terraform to manage it.**

Remember:

```
data    → look at existing thing
import  → adopt existing thing
```

10. `check` — add validation/checks

Terraform has a `check` block for assertions that don't necessarily block the entire plan/apply in the same way as preconditions.

Conceptually:

```
hcl

check "example" {
  # validation/check
}
```

Think:

`check` = **verify an assumption about your infrastructure.**

You won't need this as often as `resource`, `data`, `variable`, etc.

11. `check` **vs** `precondition` / `postcondition`

You'll sometimes see these inside resources.

For example:

```
hcl

resource "aws_instance" "app" {
  # ...

  lifecycle {
    precondition {
      condition     = var.environment != ""
      error_message = "Environment must be provided."
    }
  }
}
```

```
}  
}  
}
```

Think:

precondition = don't proceed unless this is true.

A `postcondition` checks something after the resource is evaluated/applied.

These are useful for enforcing assumptions.

12. `lifecycle` — control resource behavior

This is **not a top-level block** like `resource`; it is normally inside a resource.

Example:

```
hcl  
  
resource "aws_instance" "app" {  
  # ...  
  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```

Common lifecycle settings:

`create_before_destroy`

```
hcl  
  
lifecycle {  
  create_before_destroy = true  
}
```

Means:

Create replacement first, then destroy old resource.

Useful when downtime should be minimized.

`prevent_destroy`

```
hcl

lifecycle {
  prevent_destroy = true
}
```

Means:

Terraform should refuse to destroy this resource.

Useful for things like important production databases.

`ignore_changes`

```
hcl

lifecycle {
  ignore_changes = [
    tags
  ]
}
```

Means:

Don't try to change this property if it differs.

Useful when something outside Terraform is intentionally modifying a property.

13. `count` — create multiple copies

Example:

```
hcl

resource "aws_instance" "app" {
  count = 3

  # ...
}
```

Terraform creates:

```
aws_instance.app[0]
aws_instance.app[1]
```

Think:

`count` = **how many copies?**

14. `for_each` — create resources from a collection

Example:

```
hcl

variable "buckets" {
  default = [
    "logs",
    "backups",
    "data"
  ]
}

resource "aws_s3_bucket" "app" {
  for_each = toset(var.buckets)

  bucket = each.value
}
```

Terraform creates:

```
logs
backups
data
```

Think:

`for_each` = **create one resource for each item.**

You'll see this **a lot** in production Terraform.

15. `dynamic` — dynamically generate nested blocks

Suppose a resource has repeated nested configuration.

You can do:

```

hcl

dynamic "ingress" {
  for_each = var.rules

  content {
    from_port = ingress.value.from_port
    to_port   = ingress.value.to_port
  }
}

```

Think:

`dynamic` = **generate repeated nested blocks programmatically.**

Don't worry too much about this when you're starting.

16. `provisioner` — run commands

This is where your previous `local-exec` example comes in.

```

hcl

provisioner "local-exec" {
  command = "echo hello"
}

```

Terraform runs a command on the machine executing Terraform.

There is also:

```

hcl

provisioner "remote-exec" {
  # ...
}

```

But **provisioners are generally discouraged** when a native Terraform/provider resource can do the job.

That's why your code review complained about:

```

hcl

null_resource

```

with:

```
hcl
local-exec
```

They are essentially saying:

"Don't make Terraform run AWS CLI commands when the AWS provider already has a proper resource for this."

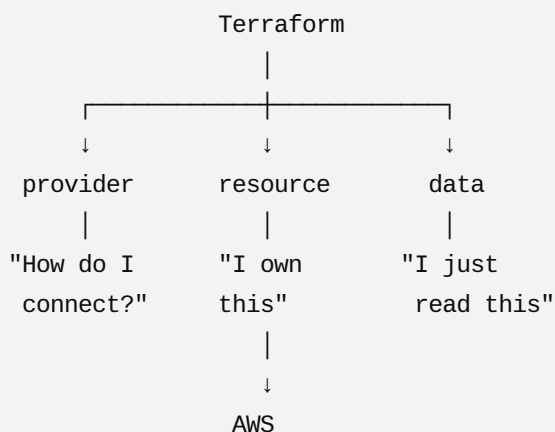
The ones you should learn first

Don't try to memorize everything above.

For your current Terraform learning, I'd learn these in this order:

- | | |
|--------------|------------------------------------|
| 1. resource | ← Terraform manages infrastructure |
| 2. data | ← read existing infrastructure |
| 3. provider | ← connect Terraform to AWS/etc. |
| 4. variable | ← inputs |
| 5. output | ← outputs |
| 6. locals | ← reusable internal values |
| 7. module | ← reusable Terraform code |
| 8. terraform | ← Terraform/provider configuration |
| 9. lifecycle | ← control resource behavior |
| 10. for_each | ← create multiple resources |
| 11. count | ← create multiple copies |
| 12. import | ← adopt existing infrastructure |

And keep this mental model:



Then around that:

variable → inputs
locals → reusable internal values
module → reusable Terraform code
output → expose useful values
import → adopt existing resources
lifecycle → control resource behavior
backend → where Terraform state lives

If you understand `resource` **VS** `data` **VS** `import` **VS** `module` **VS** `provider`, you're already past one of the biggest beginner hurdles in Terraform.

Is this conversation helpful so far?